SÃO PAULO
GOVERNO DO ESTADO
SÃO PAULO SÃO TODOS

Desenvolvimento WEB II
- ReactJS e NextJS

Prof. Tiago Arruda

O que é React?

- Biblioteca JavaScript para construção de interfaces
- Baseado em componentes reutilizáveis
- Facilita a criação de aplicações interativas e escaláveis
- Utiliza Virtual DOM para otimizar renderizações

DOM - Document Object Model

- O DOM do navegador (Document Object Model) é a representação em memória que o navegador cria de uma página HTML.
- Ele transforma o HTML em uma estrutura de árvore de objetos, que pode ser lida e modificada com JavaScript.

Virtual DOM

- Virtual DOM é uma representação virtual da DOM, mantida pelo React
  - Em vez de alterar o DOM diretamente a cada mudança de estado, o React:
    - Atualiza o Virtual DOM
    - Compara com a versão anterior
    - Aplica no DOM real apenas o que mudou

O que é React?

React não é apenas uma ferramenta, é uma mudança de mentalidade sobre como construir interfaces de usuário modernas e escaláveis.

Biblioteca vs Framework

Diferente de frameworks opinativos, o React foca exclusivamente na camada de visualização (View).
Isso oferece flexibilidade para escolher outras ferramentas conforme o projeto cresce.

Paradigma Declarativo

Você descreve *o que* a interface deve ser baseada no estado atual.
O React cuida de atualizar o DOM de forma eficiente quando os dados mudam.

Composição de UI

A interface é construída combinando pequenos componentes isolados e reutilizáveis.
Pense em blocos de Lego: peças simples formam estruturas complexas.

Next.js: visão geral

O Framework NextJs

Oferece a estrutura completa que o React puro não tem: Roteamento, Bundling e Renderização no Servidor.

Renderização Híbrida

Suporte flexível para SSR (Server-Side), SSG (Static Generation) e ISR, página por página.

App Router

O novo padrão arquitetural focado em Server Components, Layouts aninhados e Streaming de dados.

Tipos de Renderização

SSR – SERVER SIDE RENDERING

Usuário
↓
Servidor (busca dados + renderiza HTML)
↓
HTML pronto
↓
Navegador

SSG – STATIC SITE GENERATION

BUILD (deploy)
↓
HTML estático gerado
↓
CDN
↓
Usuário

ISR — Incremental Static Regeneration

BUILD
↓
HTML estático inicial
↓
Usuário 1 ——► recebe versão antiga
|
(tempo expira / revalidate)
↓
Servidor regenera em background
↓
Usuário 2 ——► recebe versão nova

Criando projeto com create-next-app

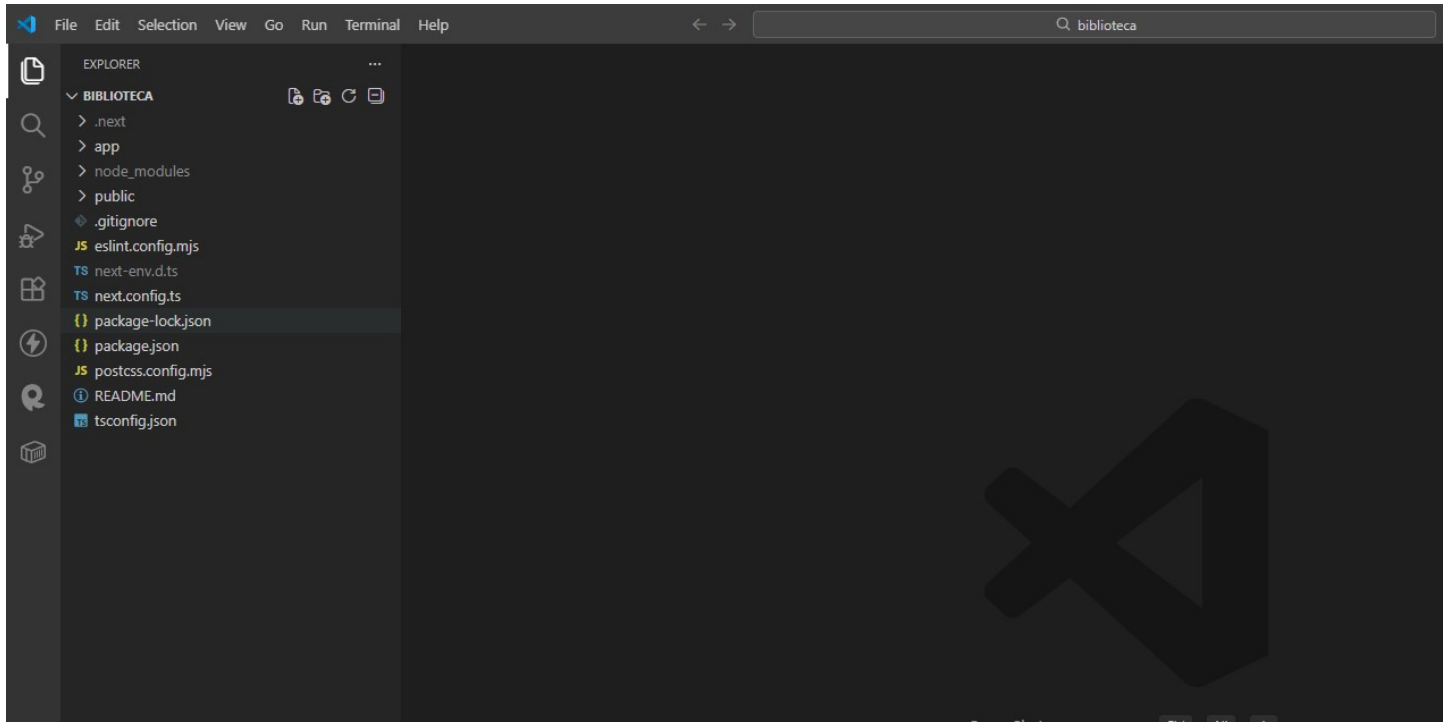
- Scaffolding Automático
  - A CLI configura TypeScript, ESLint e Tailwind CSS com perguntas interativas.
- Scripts NPM
  - Use npm run dev para desenvolvimento local com Hot Reload.
- Build de Produção
  - O comando npm run build gera arquivos otimizados e estáticos para deploy.
- Zero Config
  - Compilação e bundling (com Turbopack ou Webpack) funcionam "out of the box".

```
# No terminal:
npx create-next-app@latest biblioteca
# Output interativo:
? Would you like to use TypeScript? ›
Yes
? Would you like to use Tailwind CSS?
› Yes
? Would you like to use App Router? ›
Yes
```

Abertura do projeto no VsCode

cd .\biblioteca\ # navega para dentro do projeto biblioteca

code . # abre o diretorio atual no vscode



Conceitos de componentização

- Componentes Funcionais
  - Funções JavaScript que recebem dados e retornam elementos de UI.
- Props & Children
  - Mecanismo para passar dados de pai para filho, permitindo customização dinâmica.
- Composição
  - Agrupe componentes menores para criar seções completas de página.
- Granularidade
  - Prefira componentes pequenos com responsabilidade única a "monólitos" gigantes.

```
// Exemplo Simples
function Welcome({ name }) {
  return (
    <h1>Olá, {name}</h1>
  );
}
```

JSX: sintaxe e ideias-chave

- Sintaxe Híbrida
  - Escreva mark-up similar a HTML diretamente dentro da lógica JavaScript.
- Expressões Dinâmicas
  - Use chaves {} para interpolar variáveis e executar JS.
- Fragmentos
  - Agrupe elementos com <>...</> sem sujar o DOM com divs extras.
- Listas e Keys
  - Sempre forneça uma prop key única ao renderizar arrays.

```
// Componente com lógica e view
const user = { name: "Ana" };
return (
  <>
    <h1>Olá, {user.name}</h1>
    <ul>
      {items.map(item => (
        <li key={item.id}>
          {item.text}
        </li>
      ))}
    </ul>
  </>
);
```

Roteamento no Next.js (App Router)

- File-System Routing
  - A estrutura de pastas dentro de **app/** define automaticamente as rotas da URL.
- Arquivos Especiais
  - Use **page.tsx** para UI da rota e **layout.tsx** para UI compartilhada.
- Route Groups
  - Pastas com parênteses **(admin)** organizam código sem criar segmentos de rota.
- Loading & Error
  - Arquivos **loading.tsx** e **error.tsx** criam UIs de carregamento e fallback automaticamente.

```
// app/blog/[slug]/page.tsx
export default function Page({ params })
{
  return (
    <div>
      <h1>Post: {params.slug}</h1>
    </div>
  );
}
```

Modelos de renderização de Componentes

Server Components

O padrão no App Router

Vantagens:

- Zero JavaScript enviado ao cliente
- Acesso direto ao Backend/DB
- Melhor Performance e SEO

Use para: Busca de dados, layout estático.

Client Components

use client

Interatividade:

- Listeners (onClick, onChange)
- Estado e Efeitos (useState, useEffect)
- APIs do navegador

Use para: Formulários, botões, animações

Estado e Props: fluxo de dados

- Fluxo Unidirecional (One-Way)
  - Dados fluem sempre do componente pai para o filho via Props.
- Estado (State)
  - Dados locais mutáveis que, quando alterados, provocam uma nova renderização.
- Lifting State Up
  - Para compartilhar dados entre irmãos, mova o estado para o ancestral comum mais próximo.

```
// Hook de estado
const [count, setCount] = useState(0);
// Passando props para filhos
<Display value={count} />
<Button onClick={() => setCount(c => c
+ 1)} />
```

Estruturação de projeto React (padrões)

- Organização por Tipo
  - Agrupar arquivos pela sua natureza (components, hooks, utils). Ideal para projetos menores.
- Feature-based (Escalável)
  - Agrupar tudo relacionado a uma funcionalidade (ex: Auth, Cart) na mesma pasta.
- Separação de Preocupações
  - Mantenha a lógica de negócio em Custom Hooks e deixe os componentes focados apenas em renderizar.
- Nomenclatura
  - Use PascalCase para componentes (Button.tsx) e classes (Client.ts); e camelCase para funções (validateEmail.ts) e hooks (useAuth.ts).

```
biblioteca/
├── app/
│ ├── layout.tsx
│ ├── page.tsx
│ └── home
│   └── page.tsx
│ └── globals.css
├── components/
│ └── Navbar
│   └── Navbar.tsx
├── hooks/
│ └── useFetch.ts
├── lib/
│ └── db.ts
└── public/
```

SÃO PAULO
GOVERNO DO ESTADO
SÃO PAULO SÃO TODOS

OBRIGADO